Crew Promo Exercise 2026

Instructions

Each problem is a realistic business scenario (different industries), with **multi-part tasks** that span **basic -> intermediate -> expert**, including **analyst/BI**, **data engineering**, **modeling**, and **security** angles.

First craft the data model from the schema's listed below – some may be intentionally missing some information (like data types), so add information and craft a model as you think it would be modeled in a business scenario.

Then for each task, show your SQL that you would write to solve that problem – it will be pseudocode, but make the best effort to show what you would make each column (if you want to create fake data and test it out, try something like https://www.mockaroo.com/)

Along with each SQL, make sure you add comments explaining what you're doing and also which SQL dialect you're writing in (where it matters – i.e. SQL Server, Postgres, MySQL, etc.)

Create all code in a GitHub Repo and share prior to Interview Screen.

Reach out with any clarifying questions as needed.

Note: These questions are meant to probe at the competencies listed in the Crew Career Guidelines, this is only part of the promotion criteria and will be combined with soft skills and consulting competencies as well in assessing promotion candidates.

Problem 1 — Retail (Orders & Revenue Analytics) Schema

customers		**orders**		**order_items**		**products**
Customer_id int PK		Order_id int PK		Order_id int FK		Product_id int PK
First_name varchar(100)		Order_date timestamp		product _id int FK		Category varchar(100)
Last_name varchar(100)		Customer_id int FK		Qty int		Sku varchar(100)
Email varchar(200)		Status varchar(100)		Price numeric(18, 2)		Name varchar(100)
Region varchar(100)		Total_amount numeric(18, 2)				
Created_at timestamp						

customers ||- -o< orders: one customer has zero ord many orders
orders ||- -o< order_items: one order has zero or many order items
products ||- -o< order_items: one product appears in zero or many order items

```
-- customers(customer_id PK, first_name, last_name, email, region, created_at)
-- orders(order_id PK, order_date TIMESTAMP, customer_id FK, status, total_amount)
-- order_items(order_id FK, product_id FK, qty INT, price NUMERIC(18,2))
```

-- products(product_id PK, category, sku, name)

Questions

1. **Return the latest 10 completed orders in 2025, with customer full name and order total.**
    a. Answer: (PostgreSQL) For this query we will need to grab order id, order_date, and total_amount from orders.Then we need the customers name so we will take the first name and last name from the customers table. We need the full name so we use the || ' ' || to concat these two values together and make a new value called customers_name. Since we are using two tables of orders and customers we need to join them together with customer_id on both tables. A few filters we will want is that the orders need to be "completed" so we're going to assume that completed is the value we need for status on orders. Then we need to filter for just order_dates in the year 2025. Then we want the 10 latest orders so we will order by order_date desc so we see the newest first. Then limiting by 10.

```
select o.order_id, o.order_date, c.first_name || ' ' || c.last_name as customers_name, o.total_amount
from orders o
join customers c on o.customer_id = c.customer_id
where o.status = 'completed'
and cast(o.order_date as date) between '2025-01-01' and '2025-12-31'
order by o.order_date desc
limit 10;
```

2. **Show categories with ≥ $1M revenue in 2025 (sum from order_items).**
    a. Answer: This query will use a few key functions like group by, having, sum, and order by. We first start by selecting the category from the product table. We also need to select the revenue value we can create from the order items table where we take order item quantity times order item price to get total revenue. We also need three tables for this problem: products, orders, and order items so we have to join them based on product_id on products and product_id on order items. Then order_id on order items table with order_id on orders table. A few filters we will want is that the orders need to be "completed" so we're going to assume that completed is the value we need for status on orders. Then we need to filter for just order_dates in the year 2025. Now we have the query set we want to group by product category since our main focus is the categories. Then following this use a having sum of total revenue greater than or equal to 1000000 since we can't use a where statement for an aggregate function like sum. Then lastly having an order by statement on total_renenue and mark it with desc so we can see the highest first.

```
select p.category, sum(oi.qty * oi.price) as total_revenue
from products p
join order_items oi on p.product_id = oi.product_id
join orders o on oi.order_id = o.order_id
where o.status = 'completed'
and cast(o.order_date as date) between '2025-01-01' and '2025-12-31'
group by p.category
having sum(oi.qty * oi.price) >= 1000000
order by total_revenue desc;
```

3. **For each category, list the top 3 products by revenue in 2025 (ties allowed).**
    a. Answer: (Postgres) In this problem we will need to use two functions CTE and rank functions. We will wrap this query in a CTE in order to get the result we want. As for the query inside the wrap we start by selecting the category and product name from

the product table. We also need to select the revenue value we created from the order items table where we take order item quantity times order item price to get revenue. Lastly, in order to get the top three ranked we need to use the rank function where rank over then partition by category (since we're splitting this by category for three products in each) then order by revenue which is again just quantity times price. Again we need three tables for this problem: products, orders, and order items so we have to join them based on product_id on products and product_id on order items. Then order_id on order items table with order_id on orders table. A few filters we will want is that the orders need to be “completed” so we're going to assume that completed is the value we need for status on orders. Then we need to filter for just order_dates in the year 2025. Then group by product category and product name. Now to round this out we need to make this a CTE to get the top 3 so we do a with statement from our created ranked table as then wrap our query around in brackets. We then can select from this ranked table the product category, product name, total revenue, and rank.

Where the filter just needs to be 3 or equal to three to get the top three including ties then order by category and rank.

```
WITH ranked AS (
	select p.category, p.name, SUM(oi.qty * oi.price) AS total_revenue,
	RANK() OVER (PARTITION BY p.category ORDER BY SUM(oi.qty * oi.price) DESC) AS rnk
	from products p
	join order_items oi ON p.product_id = oi.product_id
	join orders o ON oi.order_id = o.order_id
	where o.status = 'completed'
	and cast(o.order_date as date) between ‘2025-01-01’ and ‘2025-12-31’
	group by p.category, p.name
)
select category, name, total_revenue, rnk
from ranked
where rnk <= 3
order by category, rnk;
```

4. **Pivot 2025 monthly revenue per category into 12 columns.**
    a. Answer: (Oracle SQL) (or you could just do 12 case statements for each month). Selecting the category from the product table. We also need to extract the month from order_date from the orders table and select that as well. We also need to select the revenue value we created from the order items table where we take order item quantity times order item price to get revenue. Again we need three tables for this problem: products, orders, and order items so we have to join them based on product_id on products and product_id on order items. Then order_id on order items table with order_id on orders table. A few filters we will want is that the orders need to be “completed” so we're going to assume that completed is the value we need for status on orders. Then we need to filter for just order_dates in the year 2025. After that we can use the pivot function to aggregate sum of revenue for each month number. Then order these by category.

```
select * from (
select p.category, EXTRACT(MONTH FROM o.order_date) AS month_num, oi.qty * oi.price AS revenue
from products p
join order_items oi ON p.product_id = oi.product_id
```

```
join orders o on oi.order_id = o.order_id
where o.status = 'completed'
and cast(o.order_date as date) between ‘2025-01-01’ and ‘2025-12-31’
)
PIVOT (
SUM(revenue) FOR month_num IN (1 AS jan, 2 AS feb, 3 AS mar, 4 AS apr, 5 AS may, 6 AS jun, 7 AS jul, 8 AS aug, 9 AS sep, 10 AS oct, 11 AS nov, 12 AS dec)
)
ORDER BY category;
```

5. **What indexes/partitions would you propose to speed up common filters (status, order_date, customer_id); discuss SARGability and materialized view(s) for monthly revenue.**
    a. Answer: There’s a few ways we could speed up the filters. I would index on status since the status column on the orders table likely has a few distinct values. Doing so would make it easier for the database to find the completed orders then scan the order date or any date range after. Another index I would propose would be indexing on customer_id on the orders table and order items on the orders table would help joining a lot. As far as SARGability we would ideally want an index to evaluate a condition so we’re avoiding using cast as date or lower on columns like status. As far as partitioning we would want to partition by order_date so we can easily filter by month or year. We could do a materialized view for monthly revenue where we prejoin all of the tables together so we don't have to continuous join the tables and just use a view called monthly_revenue where it ideally has columns for the order_date year and the order_date month for easy filtering and also precalculated field for total_revenue.

Problem 2 — Healthcare (Appointments & Patient Gaps) Schema

patients		**appointments**
Patient_id int PK		Appt_id int PK
name varchar(200)		Patient_id int FK
Birth_date DATE		Appt_date DATE
Created_at timestamp		Status ENUM
Region varchar(100)		

patients ||--o< appointments: one patient has zero or many appointments

-- patients(patient_id PK, name, birth_date, created_at, region)

-- appointments(appt_id PK, patient_id FK, appt_date DATE, status) -- 'SCHEDULED','COMPLETED','NO_SHOW'

Questions

1. **List patients with no appointments in the last 12 months, newest patients first.**

a. Answer: (SQL Server) For this query we want to select the patient id, patient name, and created at columns from patients. We need both tables for this query so we must join the appointments table to the patient table using patient_id but we want only the patients with no appointments. We will need a left join so that patients without a matching appointment in that window are still returned, just with NULL values in the appointment columns. Then we can reduce the appointments table down to any appointment dates in the past 12 months using dateadd and getdate. Then we need to filter only appointments status as NULL so we are not getting any appointments where it could be with a valid status value . Then we want the newest patients first so we just order by created_at and desc so the newest ones show first.

```
select p.patient_id, p.name, p.created_at
from patients p
left join appointments a on p.patient_id = a.patient_id
and a.appt_date >= dateadd(month, -12, getdate())
where a.status IS NULL
order by p.created_at desc
```

2. **Return patients who had 3 consecutive NO_SHOWs at any point.**
   a. Answer: (SQL Server) For this problem we'll need a CTE and the function LAG. We first create our window function and inside we are going to select patient_id (since both tables have patient_id it's easier to grab patient_id now since we just need appointments table), appt_date, and status. We'll then need to select and use the lag function where it will LAG the status column meaning it'll grab the previous row of status and then store it as a new column and partition by patient_id and order by appt_date so we make sure were getting the same patient and ordered by the appt_dates. We then do this again so it'll get the past two (LAG(status, 2) previous rows of status and create a new column. This makes it easy for us in the outer query to find rows where the three status, prev_1 and prev_2 are all equal to NO_SHOW. Then we can close the CTE and then select distinct patient id and patient names from the ranked table we created then join it on the patients table. Then we lastly grab where the status is NO_SHOW, the previous status is NO_SHOW, and the second previous status is NO_SHOW.

```
with ranked AS (
  select patient_id, appt_date, status,
        LAG(status, 1) OVER (PARTITION BY patient_id ORDER BY appt_date) AS prev_1,
        LAG(status, 2) OVER (PARTITION BY patient_id ORDER BY appt_date) AS prev_2
  from appointments
)
select distinct p.patient_id, p.name
from ranked r
join patients p ON r.patient_id = p.patient_id
where r.status = 'NO_SHOW'
  and r.prev_1 = 'NO_SHOW'
  and r.prev_2 = 'NO_SHOW'
```

3. **Create a procedure sp_next_appts(@Region NULL, @FromDate, @ToDate) that returns upcoming scheduled appointments (filter region if provided).**
   a. Answer: (SQL Server) For this query we want a stored procedure so we first start by creating a procedure and labeling it sp_next_appts. We need @Region, @FromDate, and @ToDate from the description of the question where @Region can be NULL. The

we can start our query by doing “as” and then “begin” where we can now select * from patients table, but we also need the appointments table so we will join by patient_id. Our first where clause is the @Region where we want to create a filter where its possible Region is NULL or an actual value. Our next filter is using the @ToDate and @FromDate where we can filter by specific dates. Then the last filter will be appointment status is SCHEDULED. Then ordering by appt_date asc so the soonest upcoming appointments come first. Then finally closing the begin block by using “end”.

```
CREATE PROCEDURE sp_next_appts
   @Region VARCHAR(100) = NULL,
   @FromDate DATE,
   @ToDate DATE
AS
BEGIN
select *
from patients p
join appointments a on p.patient_id = a.patient_id
where(@Region IS NULL OR p.region = @Region)
and a.appt_date between @FromDate and @ToDate
and a.status = 'SCHEDULED'
order by a.appt_date asc
END
```

4. **Create view vw_monthly_no_show_rate and discuss indexes (e.g., (patient_id, appt_date)), and partitioning by appt_date.**
    a. Answer: To create a view using SQL Server we first need to call the function “create view” then name it vw_monthly_no_show_rate. From here we can use “AS” then create our query for the view. Since the view is asking for a monthly no shows rate, we will want to select first the appt_date and format it as the month of the specific year. Then we want to select count(*) as total_appointments, since appt_id is the primary key we can grab the count(*) at the total_appointments. We then need to grab the no show appts, we do that by doing a sum of case when appt status is NO_SHOW count it as a 1, if not then 0. Then finally we need to put it all together and grab the no show rate where we will want to cast this as a float since itll be a decimal then we can can again the sum of case when appt status is NO_SHOW count it as a 1, if not then 0 then dividing this by count(*) to get our no show rate. Again we need both tables so we will grab from the appointments table and then join onto the patients table using the patient_id. Then last since we're using SQL Server for this problem we’ll group by format a.appt_date to get the month of a specific year. To make it easier to filter this view we could partition this view by appt_date by months so the database will only scan for specific months. So query on a month of a date will only touch that specific month of data were filtering on. Then to quickly grab patient_ids and appt_date we could index these two columns since these columns are used often for joining on patient_id and then filtering with appt_date so indexing them helps scanning for matching rows instead of scanning the whole table.

```
CREATE VIEW vw_monthly_no_show_rate AS
select
   format(a.appt_date, 'yyyy-MM') AS appt_month,
   count(*) AS total_appointments,
   sum(CASE WHEN a.status = 'NO_SHOW' THEN 1 ELSE 0 END) AS no_shows,
```

```
   CAST(sum(CASE WHEN a.status = 'NO_SHOW' THEN 1 ELSE 0 END) AS FLOAT) / COUNT(*) AS
no_show_rate
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
GROUP BY format(a.appt_date, 'yyyy-MM')
```

Problem 3 — AdTech (Event JSON, Campaign Analytics, Lateral)

Schema

users		**events**
User_id UUID / BIGINT PK		Event_id int PK
Country varchar(2)		user_id BIGINT FK
created_at TIMESTAMP		Event_time TIMESTAMP
		payload_json JSON

users ||- -< events: one user has one or many events
events ||- -o< utm_attributes: one event has zero or one utm record

```
  -- users(user_id PK, country, created_at)
-- events(event_id PK, user_id FK, event_time TIMESTAMP, payload_json JSON)
-- Example payload_json: {"utm": {"source":"search","campaign":"fall_sale"},"page":"..."}
```

Questions

1. **Count events by utm.campaign for US, CA in Q4 2025.**
    a. Answer: (SQL Server) In this query we'll need to grab attributes from the JSON payload and both events and users tables. So we want to select JSON_VALUE (to extract the value out of the JSON) for the utm.campaign. Then we need to select the country and the count as the event_count. Now since we need both tables we'll need to join them together using user_id to connect events and users. Then the filters we need to filter to the values US or CA and then we'll filter down the event time to just Q4 of 2025 which would be from October 1st to the end of December of 2025. Now we want the count of events per country so we'll want to group by utm.campaign and country. Then we'll just order them by country event_count to see the highest first.

```
select JSON_VALUE(e.payload_json, '$.utm.campaign') AS campaign, u.country, count(*) AS event_count
from events e
join users u ON u.user_id = e.user_id
where u.country IN ('US', 'CA')
and e.event_time >= '2025-10-01' and e.event_time <  '2026-01-01'
group by
   JSON_VALUE(e.payload_json, '$.utm.campaign'),
   u.country
order by
```

```
u.country,
event_count desc;
```

2. **Extract multiple JSON fields (campaign, source) via a lateral parse if engine supports it (or repeat JSON_VALUE).**
    a. Answer: (SQL Server) This query I went with repeated JSON VALUE (SQL Server) to extract campaign and source from the JSON payload. We selected both of our JSON values and then I just grabbed event_id from events in case we were to join with another table. To do this the other way we use the function CROSS JOIN LATERAL and then enclose our select of grabbing the JSON payload attributes we need of campaign and source (SELECT e.payload_json -> 'utm' ->> 'campaign' AS campaign) in the subquery.

```
select e.event_id,
        JSON_VALUE(e.payload_json, '$.utm.campaign') as campaign,
        JSON_VALUE(e.payload_json, '$.utm.source') as source
from events e
```

3. **Return the top campaign by events per country in Q4 2025.**
    a. Answer: (SQL Server) For this query we'll want to do a CTE in order to get the top campaign. We'll enclose our query with the function with ranked as () and then within the brackets we create our ranked campaigns. FIrst we select country, JSON_VALUE campaign, and the count(*) since we want the highest event count per country. We'll also need to use the rank function to get the top country in event count so we rank over then partition by country and order by count descending to get the most at the top as rank. We need country as a column so we need to join the events table with the users table by joining on user_id. Then we need to filter down the event time to just Q4 of 2025 which would be from October 1st to the end of December of 2025. Then since we want the top campaigns by country from events we'll group by country and campaign. Then enclosing this in our CTE, we can now query off of our CTE from ranked grabbing the country, campaign, and event_count where the rank is equal to 1 to get the top campaigns in event count by country.

```
with ranked as (
select u.country, JSON_VALUE(e.payload_json, '$.utm.campaign') as campaign, count(*) as event_count,
RANK() OVER (PARTITION BY u.country ORDER BY COUNT(*) DESC) AS rnk
from events e
join users u ON u.user_id = e.user_id
where e.event_time >= '2025-10-01' and e.event_time <  '2026-01-01'
group by country , JSON_VALUE(e.payload_json, '$.utm.campaign')
)
select country, campaign, event_count
from ranked
where rnk =1
```

4. **Propose a Materialized View for daily events by country/campaign with partitioning on event_time and indexes on (country, event_time).**
    a. Answer: (PostgreSQL) In order to create a materialized view you have to start with the statement 'create materialized view view_name' followed by a 'as'. Then after we can start our query for this view. We want a simple query where we can grab the daily events by country and or campaign. So we first select event_time, however we just want the day since we want daily so we simply do DATE_TRUNC and 'day' for the event_time day. We then select country and campaign using the JSON_VALUE to

grab. Then lastly select the event_count so we can get the count of events. We'll need both tables of events and users so we will join both on user_id. Then we group by our aggregate event_time as day, country, and campaign (JSON_VALUE). Then to close the materialized view we end with "WITH DATA". The second part of the question is asking about partitions on event_time so we can create a table where we partition of events from the values of Q4 which would be between 2025-10-01 to 2026-01-01. Then the next part is asking about indexing on country and event_date, we can do this by creating an index on our index name on our materialized view for (country, event_date).

```
create MATERIALIZED VIEW mv_daily_events
AS
select DATE_TRUNC('day', e.event_time) AS event_date, u.country, JSON_VALUE(e.payload_json,
'$.utm.campaign') AS campaign, count(*) AS event_count
from events e
join users u on u.user_id = e.user_id
group by
DATE_TRUNC('day', e.event_time),
u.country,
JSON_VALUE(e.payload_json, '$.utm.campaign')
WITH DATA;

CREATE TABLE events_2025_q4 PARTITION OF events
	FOR VALUES FROM ('2025-10-01') TO ('2026-01-01');
CREATE INDEX idx_country_date
	ON mv_daily_events (country, event_date);
```

Problem 4 — Finance (Ledger, ACID, Upserts, Normalization)

Schema

accounts		**ledger**		**staging_accounts**		**dim_customers**
Account_id int PK		Tax_id int PK		Account_id int		Customer_id int PK
name varchar(200)		Account_id int FK		Email varchar(200)		Email varchar(200) UNIQUE
balance numeric(18,2)		txn_time timestamp		updated _at timestamp		updated _at timestamp
		Amount numeric(18,2)				
		Currency varchar(3)				
		Memo text				

```
-- accounts(account_id PK, name, balance NUMERIC(18,2))
-- ledger(txn_id PK, account_id FK, txn_time TIMESTAMP, amount NUMERIC(18,2), currency, memo)
-- staging_accounts(account_id, email, updated_at)
-- dim_customer(customer_id PK, email UNIQUE, updated_at)

accounts        ||- -<  ledger            : one account has one or many transactions
staging_accounts  - -<  dim_customer      : staging accounts upsert into zero or many customer records (via
email)
```

Questions

1. **Atomic Transactions: Suppose your application needs to debit account 1 by $100 and credit account 2 by $100, write some sql code to update those accounts; rollback on any error.**
    a. Answer: (PostgreSQL) In order to update the table accounts we need to use the UPDATE statement. We also want to make sure there's a rollback on any error so we wrap the update's in DO $$ and $$ with an exception block after the update with rollback and raise if there's any errors. Now in order to start an update statement we want to first call BEGIN and then we can UPDATE our table accounts and set balance = balance minus 100 for debit to account 1 and the UPDATE or accounts table and set balance = balance + 100 for credit to account 2. Then we proceed with our exception block and then finally ending with the statement "END".

```
DO $$
BEGIN
	UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
	UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
EXCEPTION
	WHEN OTHERS THEN
		ROLLBACK;
		RAISE;
END;
$$;
```

2. **Assume ledger.amount can be text in some source, how would you handle that data in a safe manner to ensure it's an amount?**
    a. Answer: (PostgreSQL) We can handle this by using a specific regex. So when we query we would do a case statement when amount is not '^-?[0-9]+(\.[0-9]+)?$' meaning if the number is negative or positive with decimals then make the amount a numeric value. Otherwise make the value NULL. This way if the amount is not in that structure it'll be pushed to NULL.

```
select txn_id, account_id,
CASE
WHEN amount ~ '^-?[0-9]+(\.[0-9]+)?$' THEN amount::NUMERIC(18,2)
ELSE NULL
	END AS amount_clean
from ledger;
```

3. **Show some SQL to Upsert staging_accounts into dim_customer (newer updated_at wins) and insert any new records.**
    a. Answer: (PostgreSQL) In order to upsert a table into another table we need to use the statement "insert into". So we want to insert into email and updated_at. We then select the columns email and updated_at from our base table of staging_accounts. Then we want to use the statement on conflict (email), this will check to see if the email already exists in dim_customer. This will also add new records if there is no conflict meaning the email didn't exist in dim_customer. On conflict works on dim_customer because email in dim_customer is unique. Then we want the newer records so we use statement do update set updated_at = excluded.updated_at where excluded.updated_at is greater than dim_customer.updated_at. Using this statement gives the newer wins condition where it'll overwrite the old record from staging_accounts.

```
INSERT INTO dim_customer (email, updated_at)
select email, updated_at
```

```
from staging_accounts
ON CONFLICT (email)
DO UPDATE SET
updated_at = EXCLUDED.updated_at
where EXCLUDED.updated_at > dim_customer.updated_at;
```

b. **Bonus: How would you tweak to handle deletes if staging_accounts was a full dataset of all current accounts?**
   i. Answer: We would want to do a soft delete rather than hard delete of our records is general best practice. Instead we can create a value of curr_rcd where we mark it as FALSE if the record is no longer needed so we're not completely removing it from our data. Since staging_accounts is the full dataset of all current accounts we can update dim_customers instead. My client uses curr_rcd_ind = 'Y' for all of their current records so I'll do something similar where we add the column using alter table statement to our table dim_customer then we add column curr_rcd BOOLEAN DEFAULT TRUE where true is our default value and FALSE will be the inactive indicator. Then if the email becomes back to an active record we can alter this record back to TRUE. Then we can proceed by updating dim_customer and set curr_rcd as false where email is not in subquery select emails from staging accounts.

```
ALTER TABLE dim_customer ADD COLUMN curr_rcd BOOLEAN DEFAULT TRUE;

UPDATE dim_customer
SET curr_rcd = FALSE
where NOT EXISTS (
select 1 from staging_accounts where staging_accounts.email = dim_customer.email);
```

Problem 5 — HR / Org & Temporal / Security

Schema

employees		**comp_changes**		**users**
emp_id int PK		emp_id int FK		user_id int PK
Manager_id int FK NULL		Effective_from date		region varchar(200)
Name varchar(200)		Effective_to date		Profile_json JSON
Dept varchar(200)		salary numeric(18,2)		

employees ||--o< comp_changes: one employee has zero or many comp changes
employees ||--o| employees: one employee has zero or one manager (self-ref)
users (no relationships)

```
-- employees(emp_id PK, manager_id FK NULL, name, dept)
-- comp_changes(emp_id FK, effective_from DATE, effective_to DATE, salary NUMERIC(18,2))
-- users(user_id PK, region, profile_json JSON)
```

Questions

1. Return the chain of command with depth for each employee; prevent cycles.

a. Answer: (PostgreSQL) This is a common recursion CTE SQL problem getting the hierarchy of individuals of a company. We want to wrap this problem around a CTE and call it something like employee_hierarchy. The first part in the inside query will begin grabbing the employee id, manager id, 0 as level, and our path which will use the ARRAY statement on [emp_id] where their manager_id is null where they do not have a manager hence top of the hierarchy. Then for recursion we need to use union all and select employee id and manager id again, we also want to grab level+1 for the next level of employees from the employees table. We will also grab the combination of path and employee id. Then for recursion we need to inner join the employees table on our CTE employee_hierarchy where manager id (employees) is equal to the employee id (employee_hierarchy). Then to prevent cycles we add the filter where not employee id = any employee hierarchy path to prevent any loops. Then we can close out our CTE and query from our CTE grabbing the employee id, name, level, and path. Where we will order by path to get the order of hierarchy where the direct reports will be under the managers employee id.

```
with employee_hierarchy as (
	select emp_id, manager_id, 0 AS level, ARRAY[emp_id] AS path
	from employees e
	where manager_id is NULL
	union all
	select e.emp_id, e.manager_id, eh.level + 1, eh.path || e.emp_id
	from employees e
	inner join employee_hierarchy eh on e.manager_id = eh.emp_id
	where NOT e.emp_id = ANY(eh.path)
)
select emp_id, name, level, path
from employee_hierarchy
order by path
```

2. **What was employee 123's salary on 2025-08-15?**
   a. Answer: For this problem we need a few columns from employee and comp_changes. First we need to select employee id, name, salary, effective_from and effective_to. Then we need to join our employees table with the comp_changes table using employee id. Then we filter to just employee id 123 and then since we have the effective_from column we want to find any record of this employee where their salary has changed so any time before or on 2025-08-15. This will get us this person's active salary as of 2025-08-15. Then we need our effective to filter to be anything on or after 2025-08-15 or NULL to understand if their salary is still the same.

```
select e.emp_id, e.name, c.salary, c.effective_from, c.effective_to
from employees e
join comp_changes c on e.emp_id = c.emp_id
where emp_id = 123
and c.effective_from <= '2025-08-15' and (c.effective_to >= '2025-08-15' or c.effective_to IS NULL)
```

3. **Only allow analysts to read their region's users (Postgres example).**
   a. (Postgres) In order to limit analysts' read access to their region's user we need to enable row level security. We can do this by statement alter table (users) then enable row level security. From there we can create our policy and label it like analyst_region_policy. We do this ON the users table then we FOR SELECT TO analyst_role. Then grabbing the analyst region using the statement then setting the region to the current session region the user was assigned when logging in. We could get more in depth by using the department from the employees table if we want to

limit it for the analytics department. Also we could also FORCE row level security if we wanted to avoid any bypasses.

```
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
create POLICY analyst_region_policy
ON users
FOR SELECT
TO analyst_role
USING (region = current_setting('app.current_region'));
```

Problem 6 – Normalization and Data Modeling

Question

Given a denormalized Orders CSV with the fields listed below, propose a normalized and cleansed data model with **PK/FK**, **UNIQUE** and **CHECK** constraints (qty>0, price>=0, etc.).

Order Number - o

Order Date - o

Quantity - o

Price - p

Product Number - o,p

SKU - p

Product - p

Customer ID - c, o

Email - c

Name - c

Membership Date - c

Notes - o

customers		**products**		**orders**
Customer_id int PK UNIQUE		Product_number int PK UNIQUE		Order_number int PK UNIQUE
Name varchar(200)		SKU varchar(200) UNIQUE		Customer_id int FK
Email varchar(200)		Product_name varchar(200)		Product_number int FK
Membership_date date		Price decimal(18, 2) CHECK(price>=0)		Order_date date
				Quantity int CHECK (qty>0)
				Notes text

customers ||--o< orders: one customer has zero or many orders
products ||--o< orders: one product has zero or many orders
orders }o--|| customers: one order belongs to exactly one customer

orders }o--|| products: one order belongs to exactly one product

Tags

#tsi #crew #tsi-crew-promotion-2026